ALLIANCE ENGINEERS

Structural Inspection & BOQ Management System

Database + PHP/MySQL Project Guide
College / Semester Project

Purpose: This guide explains how the database-driven project works, how to install it in WAMP, how each PHP file connects to MySQL, how photo uploads are stored, how findings are generated, and how the BOQ/report flow works.

Demo login: admin / admin123

Technology: HTML5, CSS3, JavaScript, PHP 8+, MySQL/MariaDB, WAMP.

1. WHAT WE ARE BUILDING

The application is a web-based structural inspection management prototype. A user creates an inspection project, uploads multiple site photographs, records/document defects, generates a basic BOQ and prints a report.

Main workflow

```
USER
  |
  v
LOGIN
  |
  v
DASHBOARD
  |
  v
CREATE INSPECTION
  |
  v
UPLOAD MULTIPLE PHOTOS
  |
  v
DEMO PHOTO ANALYSIS
  |
  v
DEFECT FINDINGS
  |------ Cause
  |------ Severity
  |------ Risk
  |------ Recommendation
  |
  v
GENERATE BOQ
  |
  v
PRINTABLE REPORT
```

Important: The current analysis is a demonstration rule engine, not a certified structural-safety assessment. For real professional use, findings must be verified by a qualified engineer and the analysis module should be replaced or supplemented by a validated vision system.

2. PROJECT FOLDER STRUCTURE

```
AllianceEngineers_CollegeProject/
|
|-- index.php
|-- login.php
|-- logout.php
|-- dashboard.php
|-- project_new.php
|-- project.php
|-- upload.php
|-- analyze.php
|-- boq.php
|-- report.php
|-- database.sql
|-- CODE_MAP.md
|-- DATABASE_SYSTEM_GUIDE.pdf
|
|-- config/
|   `-- database.php
|
|-- includes/
|   |-- auth.php
|   |-- header.php
|   `-- footer.php
|
|-- assets/
|   |-- css/style.css
|   |-- js/app.js
|   `-- images/
|
|-- uploads/
`-- reports/
```

Easy rule: PHP files control the application flow. MySQL stores structured data. The uploads folder stores actual image files. CSS/JS control the browser interface.

3. INSTALL WAMP

• Install WAMP on Windows.

• Start Apache and MySQL.

• Wait until the WAMP icon is green.

• Test Apache at http://localhost/

• Open phpMyAdmin at http://localhost/phpmyadmin/

Put the extracted project folder directly inside:

```
C:\wamp64\www\AllianceEngineers_CollegeProject\
```

The application URL becomes:

```
http://localhost/AllianceEngineers_CollegeProject/
```

4. DATABASE SETUP — STEP BY STEP

• Open phpMyAdmin.

• Click Import.

• Select database.sql from the project folder.

• Click Go.

• phpMyAdmin will create the alliance_engineers database and its tables.

• Open config/database.php if your MySQL username/password is different from the WAMP default.

Default connection used by this project:

```
Host     = localhost
Database = alliance_engineers
User     = root
Password = (blank)
```

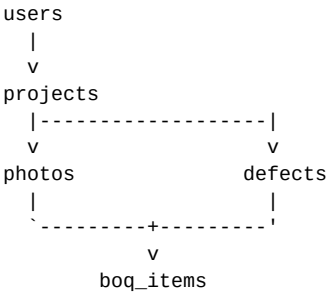
If your WAMP MySQL has a password, edit **$DB_PASS** in config/database.php.

5. DATABASE TABLES

The project uses five main tables.

Table	Purpose	Important fields
users	Login accounts	id, name, username, password_hash, role
projects	Inspection/project records	id, project_name, building_name, location, engineer_name, inspection_date
photos	Uploaded image metadata	id, project_id, file_name, original_name
defects	Inspection findings	photo_id, project_id, defect_type, severity, cause, risk, recommendation
boq_items	Bill of quantities	project_id, defect_id, code, description, quantity, unit, rate, amount

Database relationship:



6. HOW THE TABLES CONNECT

projects → **photos**: photos.project_id stores the project that owns the photo.

projects → **defects**: defects.project_id stores the project where the finding was recorded.

photos → **defects**: defects.photo_id points back to the exact uploaded photo.

defects → **boq_items**: boq_items.defect_id can identify which finding caused a BOQ item.

projects → **boq_items**: every BOQ line belongs to a project.

This is why the application can show the correct photos, findings and BOQ when you open one inspection.

Example:

```
Project #5
   |
   +-- Photo #11 -> crack.jpg
   |       |
   |       +-- Defect #7 -> Surface / Wall Crack
   |                        |
   |                        +-- BOQ #3 -> Crack repair
   |
   +-- Photo #12 -> damp.jpg
           |
           +-- Defect #8 -> Water Seepage
                                |
                                +-- BOQ #4 -> Waterproofing
```

7. WHAT EACH CODE FILE DOES

File	What it does
index.php	Entry point; redirects to login or dashboard.
login.php	Login form, user query, password verification and session creation.
logout.php	Destroys the PHP session and returns to login.
dashboard.php	Counts projects/photos/defects and displays recent inspections.
project_new.php	Form that inserts a new record into projects.
project.php	Displays one project, its photos and findings.
upload.php	Validates image uploads, saves files and inserts photo metadata.
analyze.php	Creates demo rule-based findings and saves them in defects.
boq.php	Maps findings to example work items, quantities and rates.
report.php	Combines project, findings and BOQ into a printable report.
config/database.php	PDO connection and common application constants.
includes/auth.php	Stops logged-out users from accessing protected pages.
includes/header.php	Shared navigation and page start.
includes/footer.php	Shared footer and JavaScript include.
style.css	Visual design, responsive layout and print styling.
app.js	Small browser interaction for photo uploads.
database.sql	Creates database, tables and demo records.

8. LOGIN CODE — HOW IT WORKS

login.php receives the username and password from the browser. It uses a prepared statement to find the user.

```
$stmt = $pdo->prepare(
    "SELECT * FROM users WHERE username=? LIMIT 1"
);
$stmt->execute([$username]);
$user = $stmt->fetch();
```

Then PHP verifies the stored password hash:

```
if ($user && password_verify($password, $user['password_hash'])) {
    $_SESSION['user_id'] = $user['id'];
    $_SESSION['user_name'] = $user['name'];
    header("Location: dashboard.php");
    exit;
}
```

Viva explanation: A session stores the fact that the user is logged in. Protected pages include includes/auth.php, which checks the session before showing the page.

9. PHOTO UPLOAD CODE — HOW IT WORKS

The browser sends multiple files to upload.php. The server checks the extension and image validity, generates a unique filename and moves the file into uploads/.

```
$allowed = ['jpg','jpeg','png','webp'];

$ext = strtolower(
    pathinfo($original, PATHINFO_EXTENSION)
);

if (!in_array($ext, $allowed, true)) continue;

if (@getimagesize($tmp) === false) continue;

$safe = bin2hex(random_bytes(10)) . "." . $ext;

move_uploaded_file(
    $tmp,
    UPLOAD_DIR . $safe
);
```

The filename is then stored in MySQL so the website knows which physical image belongs to which project.

10. ANALYSIS MODULE

For the semester prototype, analyze.php cycles through predefined inspection templates. Each photo receives a demonstration finding such as a crack, dampness, concrete deterioration or paint/plaster deterioration.

Each finding stores:

• Defect type

• Severity: High / Medium / Low

• Possible cause

• Risk if untreated

• Recommended solution

• Suggested timeline

• Engineer review status

This design is modular. Later, the same page can call a validated computer-vision service and save the returned structured result into the defects table.

11. BOQ MODULE

boq.php converts documented findings into example BOQ lines. This demonstrates the database and calculation workflow.

```
quantity × rate = amount

Example:
20 Sq.m × ■650 = ■13,000
```

The database uses a generated amount column so the amount is calculated from quantity and rate.

Important: Example quantities and rates are for a college demonstration. Real work should use actual site measurements and applicable approved rate/DSR data.

12. REPORT MODULE

report.php collects the project record, defects joined to their source photos, and BOQ items, then presents them as one printable inspection report.

Use the browser's Print / Save as PDF button to demonstrate the final deliverable.

The report contains:

• Project identification

• Building and location

• Engineer name and inspection date

• Photographic findings

• Severity and risk

• Recommendations

• BOQ table

• Estimated total

• Academic/professional-use disclaimer

13. COMPLETE DEMO — COLLEGE PRESENTATION

1. Login with admin / admin123.
2. Show the dashboard statistics.
3. Click New Inspection.
4. Create a sample building inspection.
5. Upload 4–8 building photographs.
6. Show that the photos appear inside the website.
7. Click Analyze Photos.
8. Explain one finding: problem → possible cause → risk → solution → timeline.
9. Click Generate BOQ.
10. Explain quantity × rate = amount.
11. Open View Report.
12. Use Print / Save as PDF.
13. Explain the database relationships in phpMyAdmin.

14. TROUBLESHOOTING

Database connection failed

Check MySQL is running and verify config/database.php.

404 / project not found

Make sure the folder is directly inside C:\wamp64\www\.

Login fails

Re-import database.sql so the current demo password hash is installed. Demo credentials are admin / admin123.

Photos don't upload

Check the uploads folder exists and PHP upload limits allow the selected files.

Images show broken

Check that uploaded files exist in uploads/ and BASE_URL in config/database.php matches the folder name.

15. VIVA QUESTIONS TO LEARN

What is PHP?
A server-side scripting language used here to process forms, sessions and database operations.

What is MySQL?
A relational database management system used to store structured project data.

What is PDO?
PHP Data Objects, a PHP interface used here to connect to MySQL and run prepared statements.

What is a primary key?
A field that uniquely identifies a row, such as id.

What is a foreign key?
A field that links one table to another, such as photos.project_id.

What is CRUD?
Create, Read, Update and Delete.

Why use prepared statements?
They separate SQL from user values and reduce SQL-injection risk.

Where are images stored?
The actual files are stored in uploads/, while filenames and project relationships are stored in the photos table.

Is the current AI certified?
No. It is a demonstration analysis engine. Professional structural decisions require qualified engineer verification.

What is BOQ?
Bill of Quantities: a structured list of work items, quantities, units and rates used for estimating work.

16. NEXT DEVELOPMENT PHASE

• Add real user roles: administrator, engineer, viewer.

• Add engineer review/approval for findings.

• Integrate a validated computer-vision model/API.

• Add image annotations for crack/damage regions.

• Add actual project measurement entry.

• Import applicable DSR/rate data with proper versioning.

• Export PDF and Excel.

• Add audit logs.

• Add secure cloud deployment and backups.

Semester-project advice: First make the complete local workflow work reliably. Then improve the UI and only after that add advanced AI. A working, understandable system is much easier to demonstrate and defend in a viva.